MLOps Assignment 1 - Heart Disease Risk Prediction

Name: Bhanu Pratap Singh **BITS ID:** 2025AE05693 **Course:** MLOps (AIMLCZG523)

Code repository: https://github.com/Thakurbhanupratap/mlops

1. Problem and dataset

The goal is to predict whether a patient has heart disease from routine clinical measurements and to serve that prediction through a monitored, containerised API. Data is the UCI Heart Disease (Cleveland) set: 303 rows, 13 features, original target 0-4 collapsed to binary (0 = no disease, 1 = disease). Class counts: 164 no disease, 139 disease (46 % positive), so the classes are close to balanced and accuracy is a reasonable headline metric; recall is tracked because missing a sick patient is the costly error.

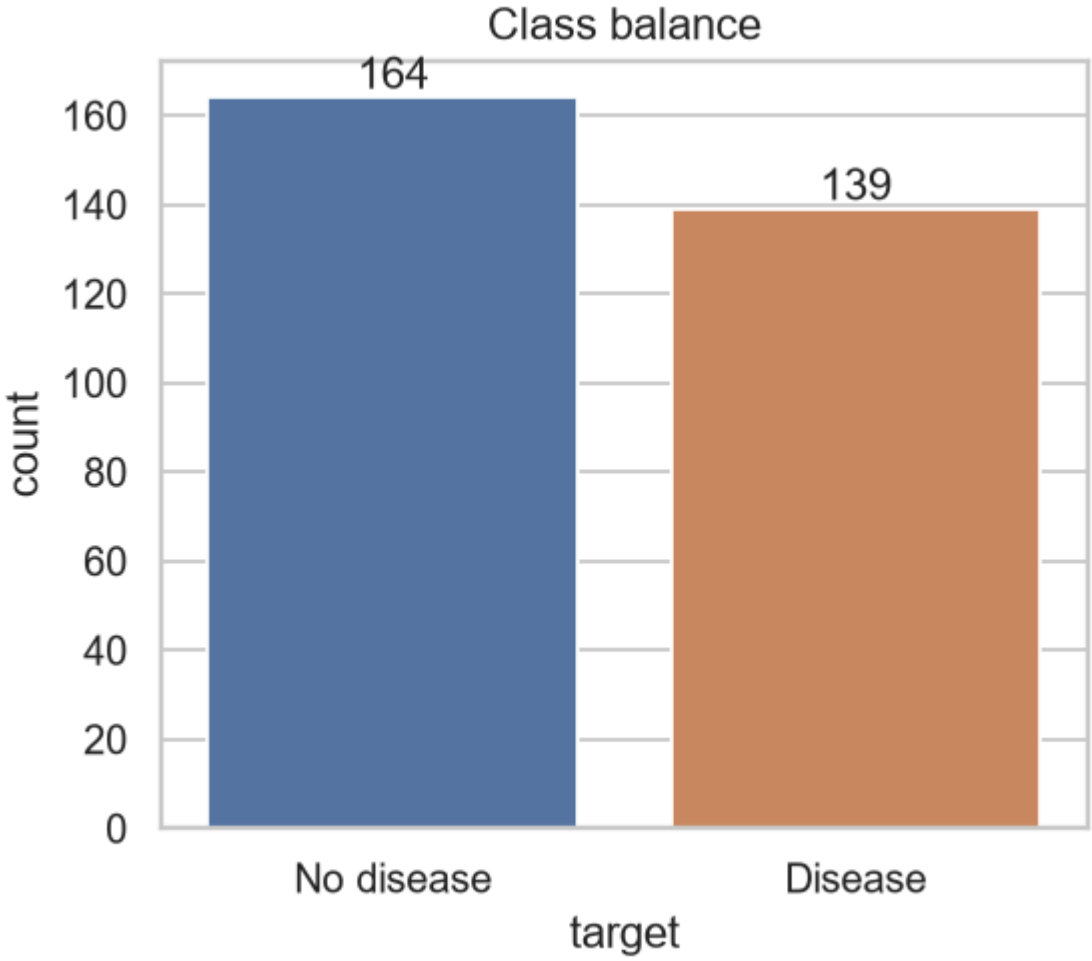
2. Setup

See README.md. Summary: `pip install -r requirements.txt`, then `python scripts/download_data.py`, `python -m src.train`, `pytest`.

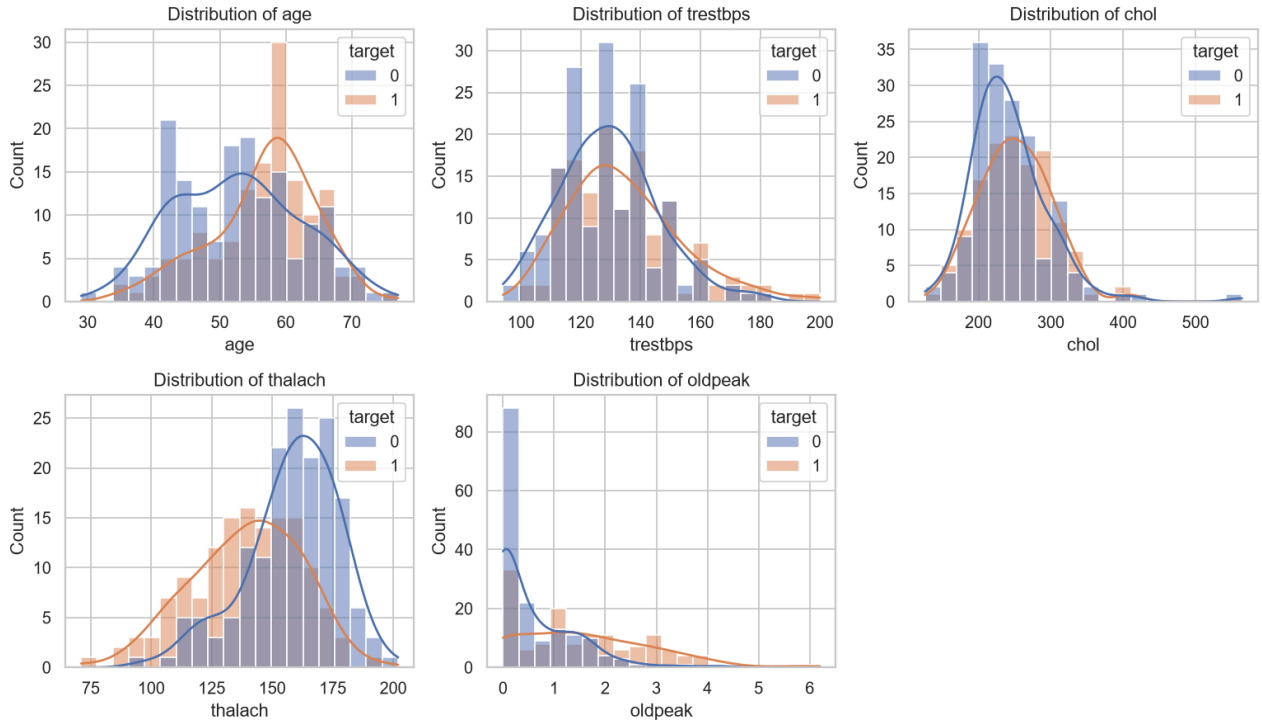
3. Data cleaning and EDA

- Raw file uses `?` for missing values. Six values are missing (4 in `ca`, 2 in `thal`); both are categorical so they were filled with the column mode.
- Target binarised; categorical columns cast to integer codes; duplicates dropped (none found).
- All plots are produced by `scripts/eda.py` and saved in `reports/figures/`.

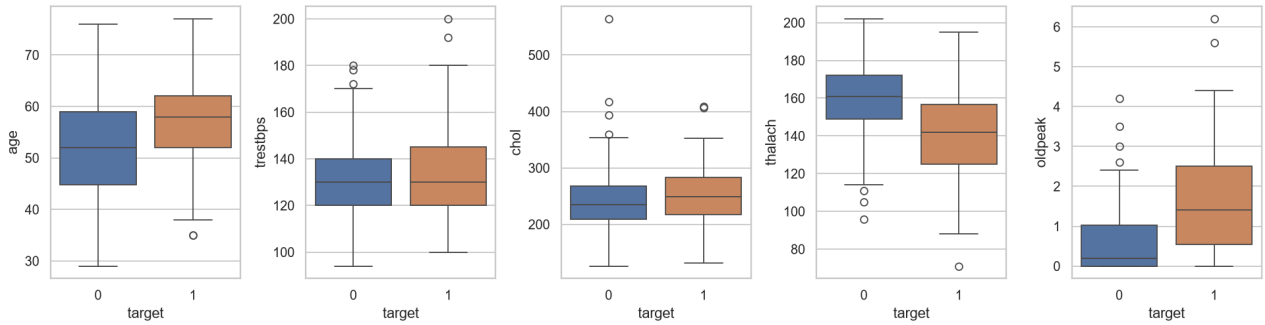
Class balance. The two classes are close to even (164 vs 139), so no resampling was needed.



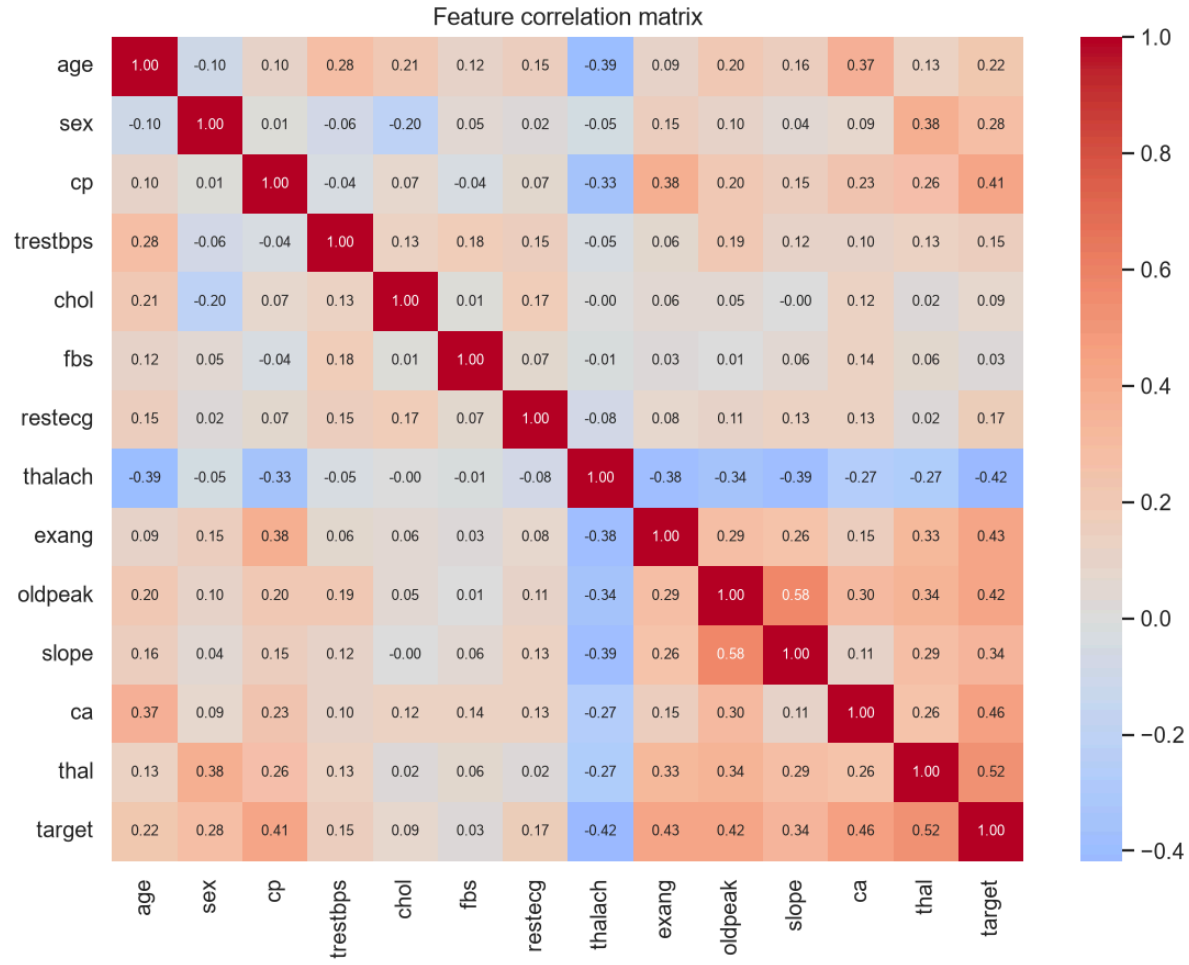
Numeric feature distributions. Patients with disease tend to have a lower maximum heart rate (`thalach`) and a larger ST depression (`oldpeak`); most healthy patients have `oldpeak` near 0. The disease group is concentrated around age 55-60, while `chol` and `trestbps` overlap heavily between the classes, so they should carry little signal on their own. `chol` has one extreme value (~560), which was kept because tree models and the scaler cope with it.



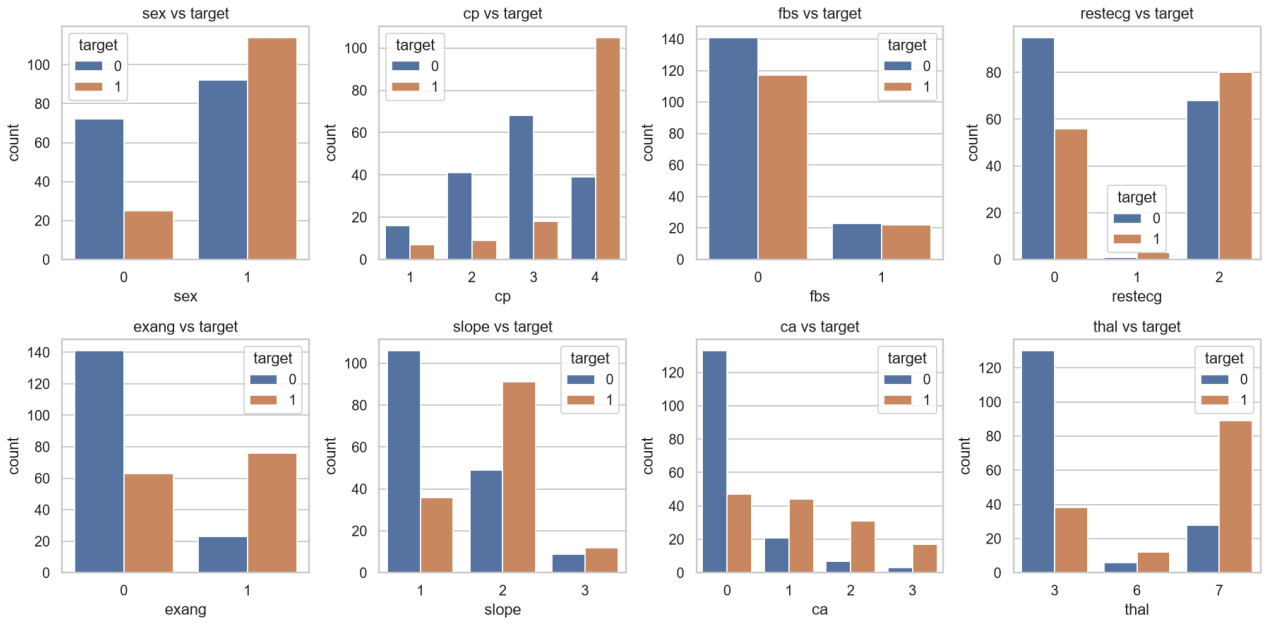
Boxplots by target. The same pattern is visible as medians: lower `thalach` and higher `oldpeak` for the disease class.



Correlation heatmap. The features most correlated with the target are `thal` (0.52), `ca` (0.46), `exang` (0.43), `oldpeak` (0.42), `cp` (0.41) and `thalach` (-0.42). `chol` (0.09) and `fbs` (0.03) are almost uncorrelated with it. `oldpeak` and `slope` are correlated with each other (0.58), which is mild multicollinearity and is the reason the L2-regularised logistic regression is a reasonable choice.



Categorical features vs target. Chest pain type 4 (asymptomatic), exercise-induced angina, `thal` = 7 (reversible defect) and a non-zero count of major vessels (`ca`) are all strongly associated with disease. Men make up most of the disease cases in this sample.



4. Features and modelling

All preprocessing lives inside one scikit-learn `Pipeline`, so the exact same transformations are applied in training and serving:

- Numeric (`age, trestbps, chol, thalach, oldpeak`): `StandardScaler`
- Categorical (`sex, cp, fbs, restecg, exang, slope, ca, thal`): `OneHotEncoder(handle_unknown="ignore")`

Data split: 80/20 stratified (random_state 42). Hyper-parameters tuned with 5-fold stratified `GridSearchCV` on the training set, optimising ROC-AUC. The test set was only used for the final reported numbers.

Model	Best params	CV acc	CV precision	CV recall	CV ROC-AUC	Test acc	Test recall	Test ROC-AUC
Logistic Regression	C=1	0.847	0.877	0.783	0.902	0.885	0.929	0.966
Random Forest	400 trees, depth 5, min leaf 3	0.814	0.831	0.755	0.904	0.852	0.893	0.955

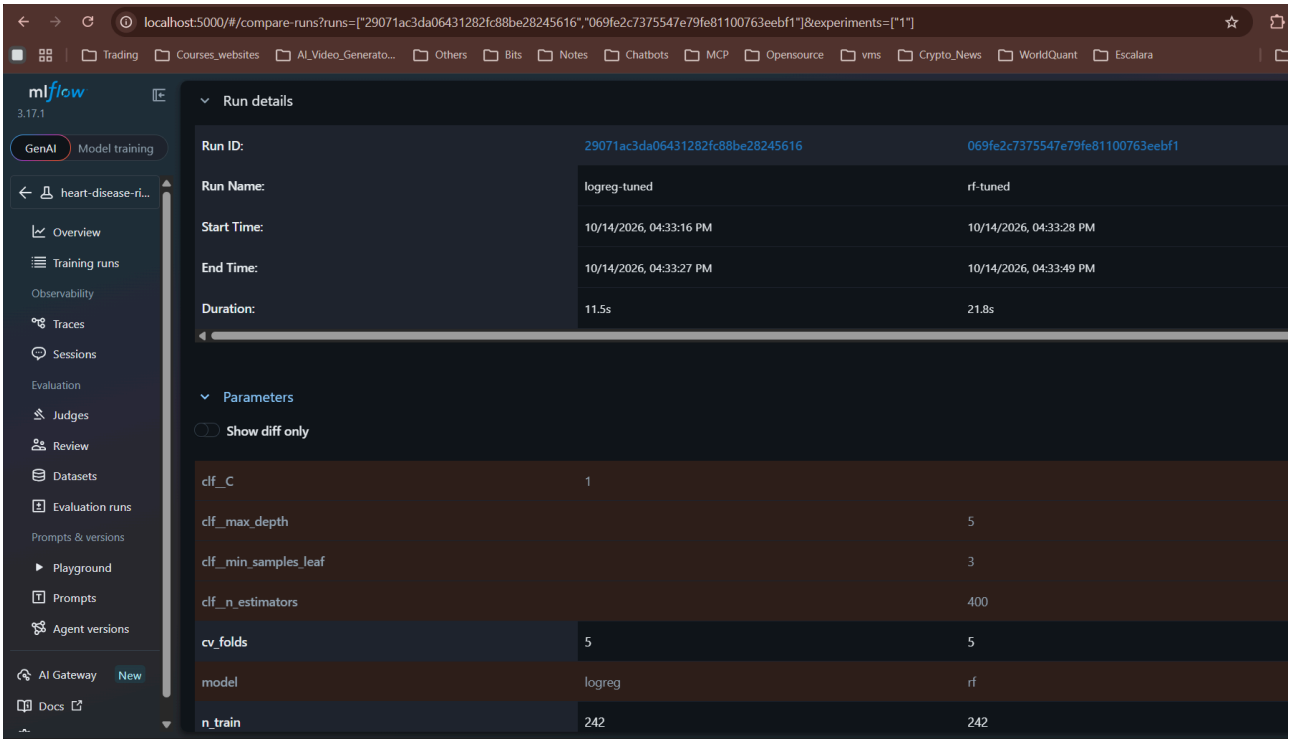
Random Forest has marginally the highest CV ROC-AUC (0.904 vs 0.902) so it is the model selected automatically by `src/train.py` and packaged. The difference is within noise (CV std is ~0.03-0.04); logistic regression is simpler and equally good, which is worth noting as a trade-off. On the held-out test set (61 patients) the Random Forest confusion matrix shows 27 true negatives, 25 true positives, 6 false positives and 3 false negatives (see section 5).

5. Experiment tracking (MLflow)

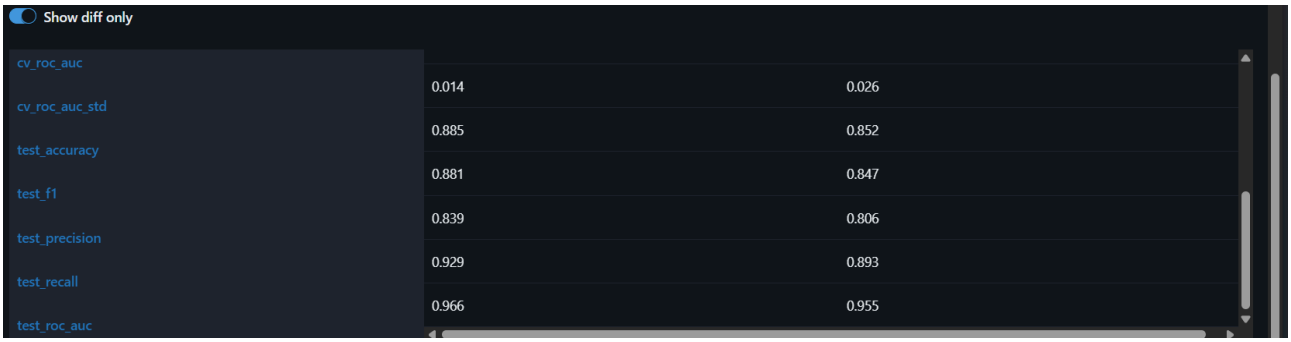
Each run logs: parameters (model, CV folds, best hyper-parameters), CV and test metrics, confusion matrix and ROC plots (artifacts), and the fitted pipeline as an MLflow model. Backend is a local SQLite DB (`mlflow.db`) with artifacts in `mlartifacts/`. View with `mlflow ui --backend-store-uri sqlite:///mlflow.db`.

Two runs were logged in the `heart-disease-risk` experiment: `logreg-tuned` and `rf-tuned`.

Run comparison: parameters. The best hyper-parameters found by grid search for each model (logistic regression `C=1`; random forest 400 trees, depth 5, min leaf 3).



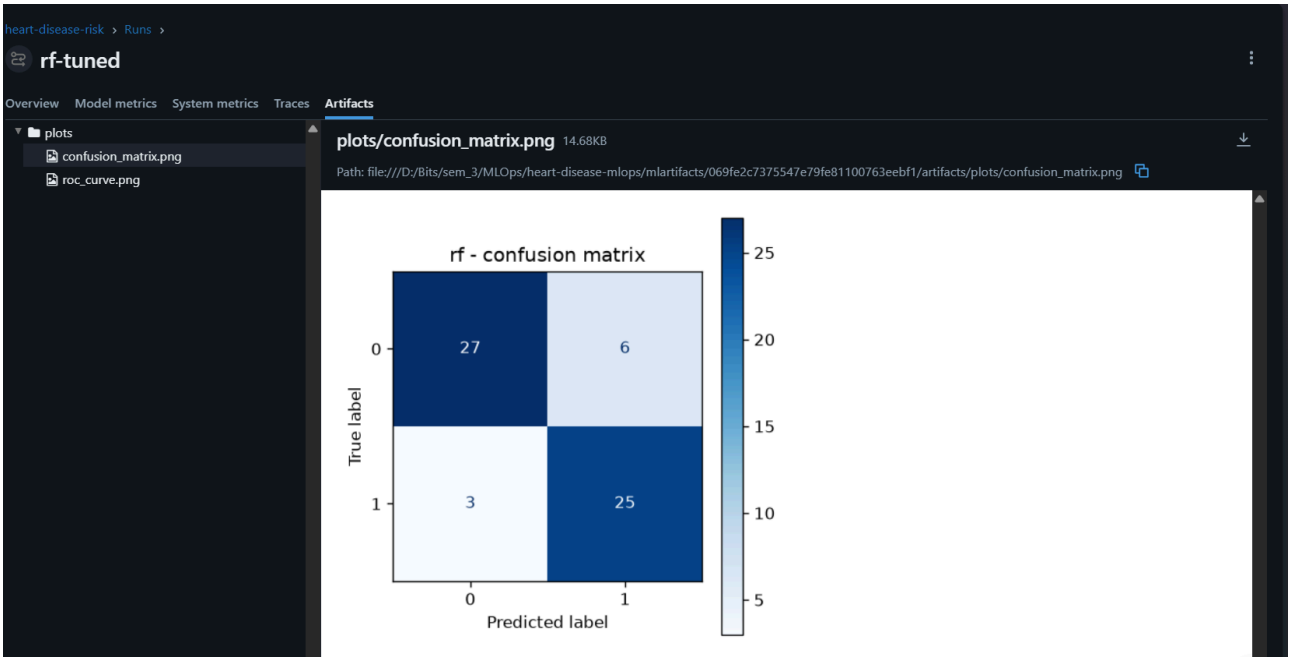
Run comparison: metrics. Test accuracy, precision, recall, F1 and ROC-AUC side by side.

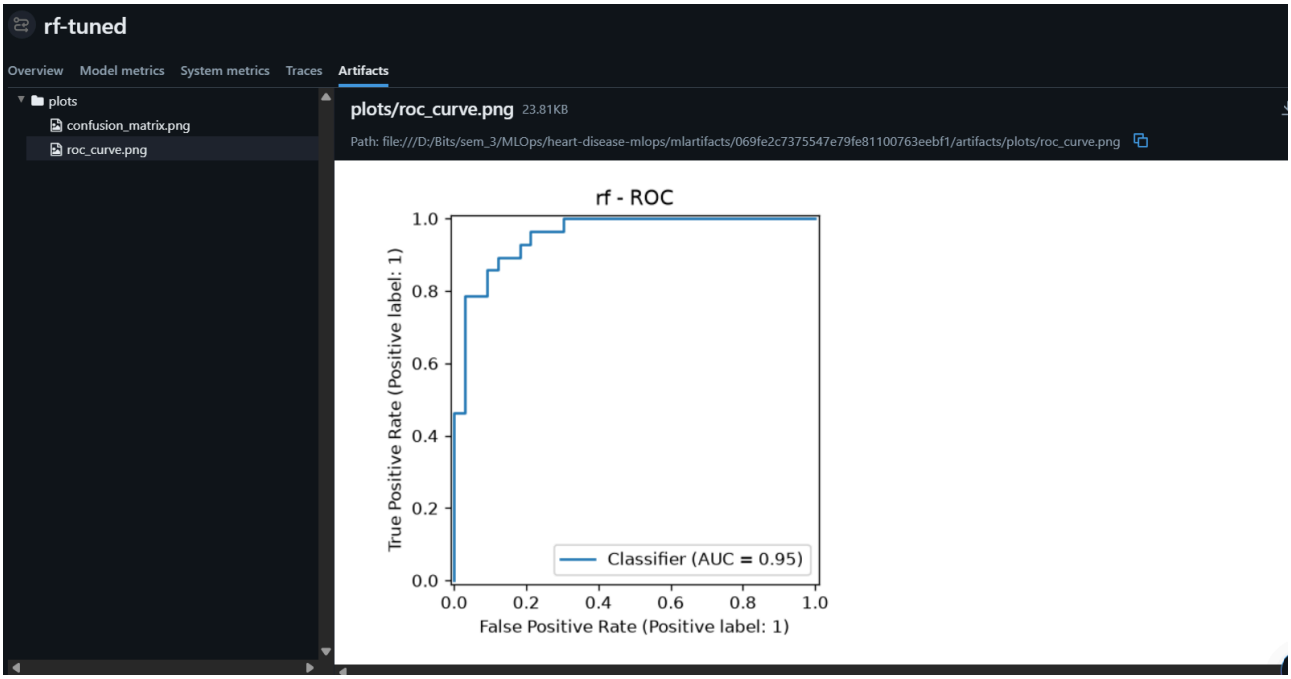


Parallel coordinates view of the tuned parameters against cross-validated accuracy.



Logged artifacts for the random forest run: confusion matrix and ROC curve (AUC 0.95).





6. Packaging and reproducibility

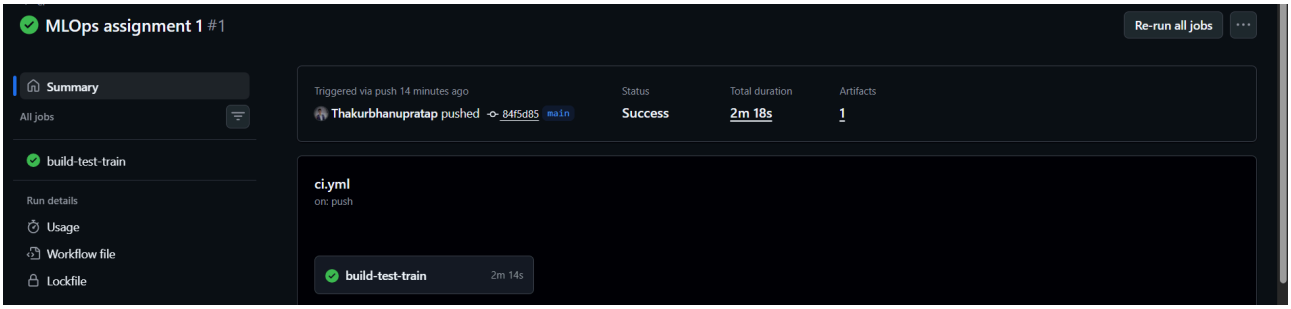
- Final pipeline saved as `models/heart_pipeline.joblib` (preprocessing + classifier in one object) and also logged to MLflow.
- `requirements.txt` pins all training dependencies; `requirements-serve.txt` is the slim set used in the Docker image.
- Random seeds are fixed in `src/config.py`.

7. Testing and CI/CD

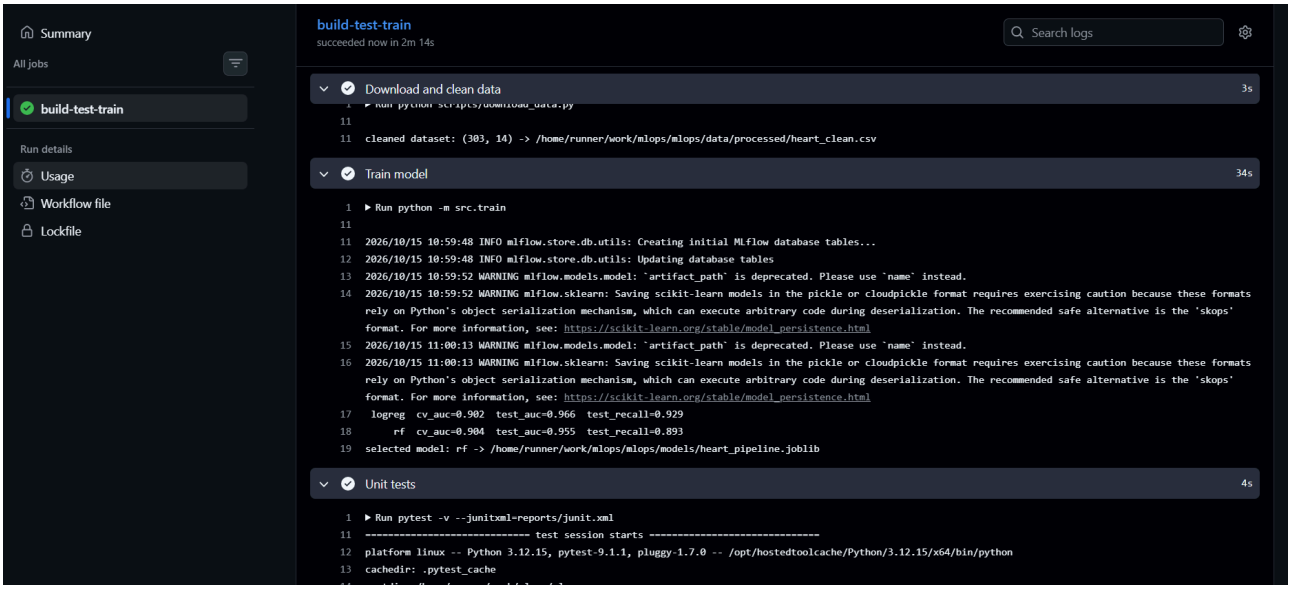
- `tests/` contains 15 tests: data cleaning, split, pipeline fitting, unseen-category handling, metrics, inference schema, and API endpoints (health, valid prediction, missing field, invalid `thal`, metrics).

- `.github/workflows/ci.yml` runs on every push/PR: install -> flake8 -> download data -> train -> pytest -> build Docker image and smoke-test `/health` and `/predict` -> upload models/reports/MLflow data as artifacts. Any lint, test or build failure fails the workflow.

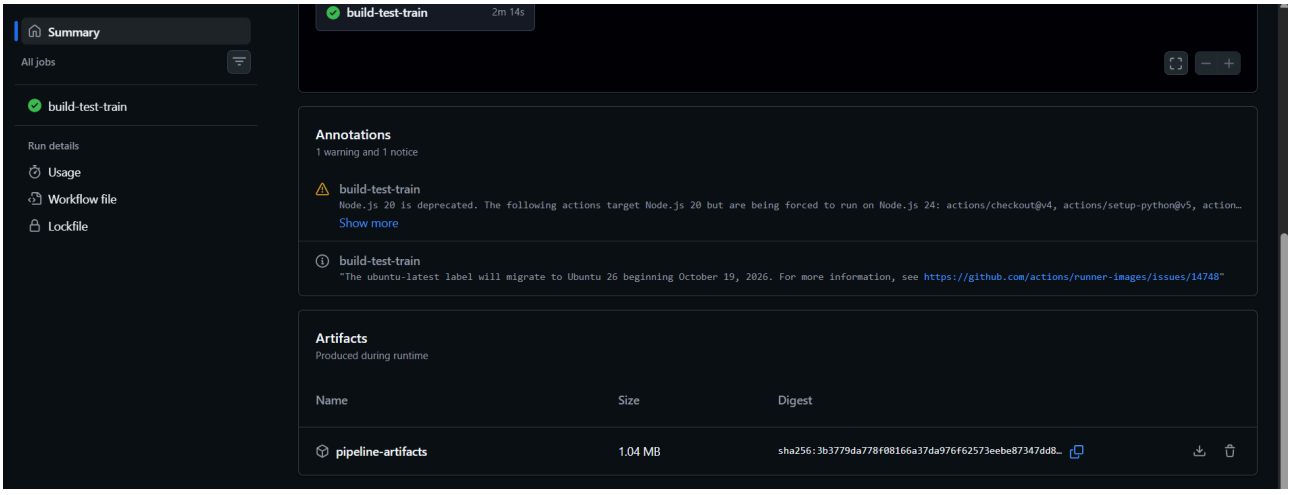
The workflow run succeeded in about 2 minutes 18 seconds and uploaded one artifact bundle (`pipeline-artifacts` , 1.04 MB) containing the model, reports and MLflow data.



Step logs: data download, model training (selected model: rf) and unit tests.



Artifacts produced by the run.



8. Containerisation

`Dockerfile` (python:3.12-slim, non-root user, healthcheck) serves `api.main:app` with uvicorn.

`POST /predict` accepts the 13 features as JSON and returns `prediction`, `confidence` and `probability_disease`. Inputs are validated by pydantic (ranges, `thal` in {3, 6, 7}).

Build. `docker build -t heart-api:1.0 .` completes and tags the image (660 MB).

```
PS>docker build -t heart-api:1.0 .
docker : #0 building with "desktop-linux" instance using docker driver
At line:1 char:1
+ docker build -t heart-api:1.0 .
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: (#0 building wit...g docker driver:String) [], RemoteException
    + FullyQualifiedErrorId : NativeCommandError

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 555B done
#1 DONE 0.0s
#2 [internal] load metadata for docker.io/library/python:3.12-slim
#2 DONE 1.0s
#3 [internal] load .dockerignore
#3 transferring context: 129B done
#3 DONE 0.0s
#4 [1/8] FROM docker.io/library/python:3.12-slim@sha256:a6e34c598f2467ed0e9a8d349809fcd8b5c603269512df273a0bb1784edc11b1
#4 resolve docker.io/library/python:3.12-slim@sha256:a6e34c598f2467ed0e9a8d349809fcd8b5c603269512df273a0bb1784edc11b1 0.0s
done
#4 DONE 0.0s
#5 [internal] load build context
#5 transferring context: 1.08kB done
#5 DONE 0.0s
#6 [4/8] RUN pip install --no-cache-dir -r requirements-serve.txt
#6 CACHED
#7 [7/8] COPY models/heart_pipeline.joblib ./models/heart_pipeline.joblib
#7 CACHED
#8 [2/8] WORKDIR /app
#8 CACHED
#9 [3/8] COPY requirements-serve.txt .
#9 CACHED
#10 [5/8] COPY src ./src
#10 CACHED
#11 [6/8] COPY api ./api
#11 CACHED
#12 [8/8] RUN useradd -m appuser
#12 CACHED
#13 exporting to image
#13 exporting layers done
#13 exporting manifest sha256:bae251e94c37bb518874fe5dd9c7e669aafe642eac90ea8517b00a830b5c572e done
#13 exporting config sha256:9fd9149f8dfd86a4075cf983c051e3e92e4a4edc6525e4c487a6e4757fe2b411 done
#13 exporting attestation manifest sha256:8a08a712221675031dca930e2a99251a71972c48fdea73723e9b59dc54191f1a
#13 exporting attestation manifest sha256:8a08a712221675031dca930e2a99251a71972c48fdea73723e9b59dc54191f1a 0.0s done
#13 exporting manifest list sha256:3793960f789201887c3561fd5103dae0467208e6e2fc440029691ecd601aae36 0.0s done
#13 naming to docker.io/library/heart-api:1.0 done
#13 unpacking to docker.io/library/heart-api:1.0 0.0s done
#13 DONE 0.1s
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/ll9c3gti6quijg7n8vb0vbsf7
```

```
PS>docker images heart-api
REPOSITORY   TAG       IMAGE ID       CREATED          SIZE
heart-api    1.0       3793960f7892   37 minutes ago   660MB
```

Run. The container starts, loads the model on startup and listens on port 8000.

```
PS>docker run -d -p 8000:8000 --name heart heart-api:1.0
298e77f50f37dda323a6c3971e7c3fb7567051fa0d7c56baad84aa6592436400
PS>docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED         STATUS                            PORTS
    NAMES
298e77f50f37   heart-api:1.0   "uvicorn api.main:ap…"   9 seconds ago   Up 8 seconds (health: starting)   0.0.0.0:8000->8000/tc
p   heart
PS>docker logs heart
docker : INFO:     Started server process [1]
At line:1 char:1
+ docker logs heart
+ ~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: (INFO:     Started server process [1]:String) [], RemoteException
    + FullyQualifiedErrorId : NativeCommandError

INFO:     Waiting for application startup.
2026-10-15 11:21:11,883 INFO heart-api model loaded
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Prediction request. A sample patient posted to `/predict` returns the predicted class, the confidence of that prediction and the probability of disease.

```
Thakur Bhanu@Thakur MINGW64 /d/Bits/sem_3
$ curl -X POST localhost:8000/predict -H "Content-Type: application/json" \
  -d '{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,"fbs":1,"restecg":2,"thalach":150,"exang":0,"oldpeak":2.3,"slope":3,"
a":0,"thal":6}'
{"prediction":0,"confidence":0.6292,"probability_disease":0.3708}
```

9. Deployment

The API is deployed locally as containers with Docker Compose (`docker-compose.yml`): the API container (built from the project `Dockerfile`), Prometheus and Grafana run together on one Docker network and are reachable on ports 8000, 9090 and 3000 (screenshots in sections 8 and 10).

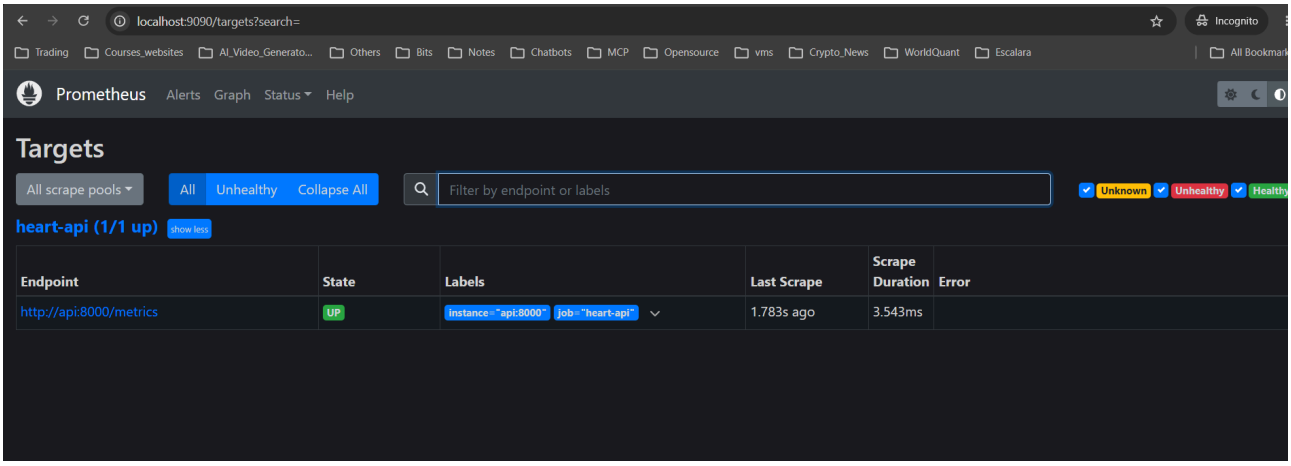
Access instructions for local testing: `docker compose up --build`, then `POST http://localhost:8000/predict` (interactive docs at `http://localhost:8000/docs`).

Kubernetes manifests are included in `k8s/` (Deployment with 2 replicas and health probes, a LoadBalancer Service and an optional Ingress) but they were not deployed to a cluster for this submission.

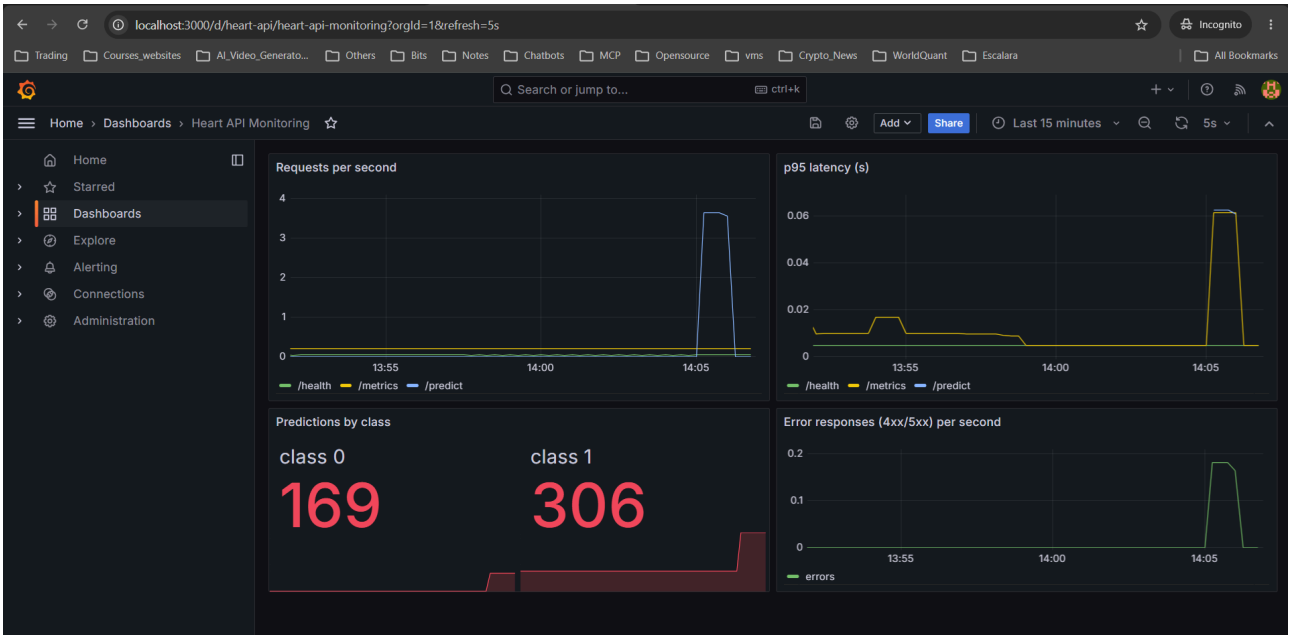
10. Monitoring and logging

- Every request is logged (method, path, status, latency) and prediction results are logged.
- `/metrics` exposes Prometheus metrics: `api_requests_total`, `api_request_latency_seconds`, `model_predictions_total`.
- `docker compose up` starts the API, Prometheus and Grafana with a pre-provisioned dashboard (request rate, p95 latency, predictions per class, error rate).

Prometheus scrapes the API every 5 seconds; the `heart-api` target is UP.



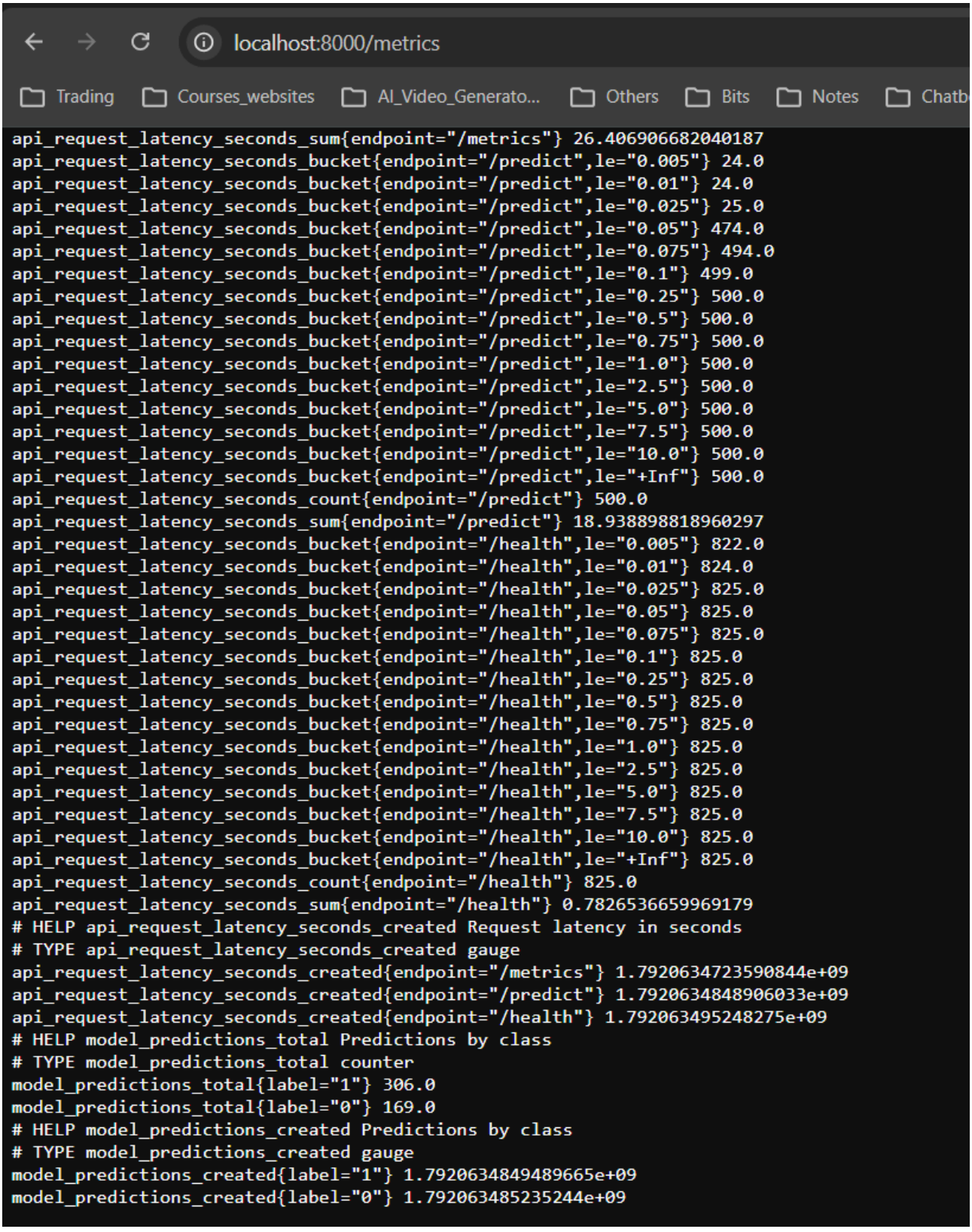
Grafana dashboard after running `scripts/load_test.py`: request rate per endpoint, p95 latency, predictions per class and the rate of 4xx/5xx responses (the load script sends some invalid requests on purpose, which is why the error panel shows a spike).



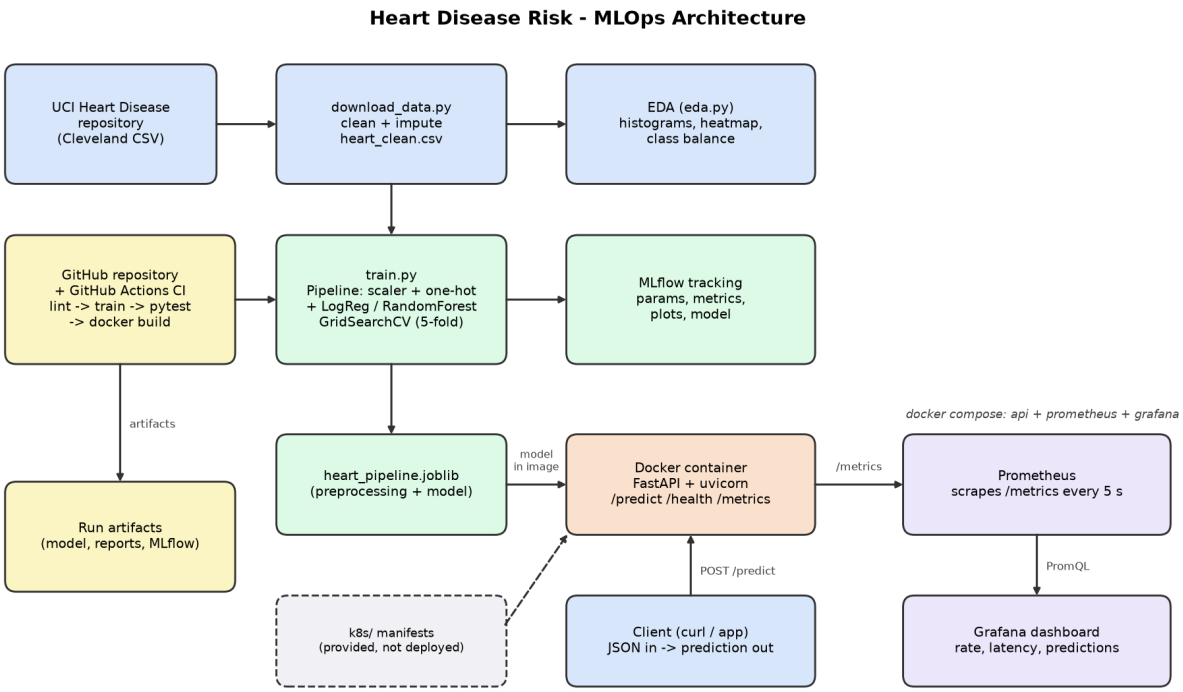
API logs. Each request is logged with method, path, status code and latency.

```
Thakur Bhanu@Thakur MINGW64 /d/Bits/sem_3/MLOps/heart-disease-mlops (main)
$ docker compose logs api | tail -20
api-1  | 2026-10-16 08:37:00,457 INFO heart-api GET /metrics -> 200 (2.2 ms)
api-1  | INFO:     172.18.0.3:54642 - "GET /metrics HTTP/1.1" 200 OK
api-1  | 2026-10-16 08:37:05,459 INFO heart-api GET /metrics -> 200 (2.6 ms)
api-1  | INFO:     172.18.0.3:54642 - "GET /metrics HTTP/1.1" 200 OK
api-1  | 2026-10-16 08:37:10,459 INFO heart-api GET /metrics -> 200 (2.1 ms)
api-1  | INFO:     172.18.0.3:54642 - "GET /metrics HTTP/1.1" 200 OK
api-1  | 2026-10-16 08:37:15,456 INFO heart-api GET /metrics -> 200 (1.6 ms)
api-1  | INFO:     172.18.0.3:54642 - "GET /metrics HTTP/1.1" 200 OK
api-1  | 2026-10-16 08:37:17,483 INFO heart-api GET /health -> 200 (0.6 ms)
api-1  | INFO:     127.0.0.1:46848 - "GET /health HTTP/1.1" 200 OK
api-1  | INFO:     172.18.0.3:54642 - "GET /metrics HTTP/1.1" 200 OK
api-1  | 2026-10-16 08:37:20,457 INFO heart-api GET /metrics -> 200 (1.8 ms)
api-1  | 2026-10-16 08:37:25,456 INFO heart-api GET /metrics -> 200 (1.6 ms)
api-1  | INFO:     172.18.0.3:54642 - "GET /metrics HTTP/1.1" 200 OK
api-1  | 2026-10-16 08:37:30,458 INFO heart-api GET /metrics -> 200 (2.8 ms)
api-1  | INFO:     172.18.0.3:54642 - "GET /metrics HTTP/1.1" 200 OK
api-1  | 2026-10-16 08:37:35,457 INFO heart-api GET /metrics -> 200 (2.3 ms)
api-1  | INFO:     172.18.0.3:54642 - "GET /metrics HTTP/1.1" 200 OK
api-1  | 2026-10-16 08:37:40,455 INFO heart-api GET /metrics -> 200 (1.6 ms)
api-1  | INFO:     172.18.0.3:54642 - "GET /metrics HTTP/1.1" 200 OK
```

Raw metrics exposed at `/metrics`.



11. Architecture



The data flows from the UCI repository through cleaning and EDA into training. Training logs every run to MLflow and saves one pipeline file, which is baked into the Docker image. GitHub Actions re-runs lint, training, tests and the Docker build on every push. The running container exposes `/metrics`, which Prometheus scrapes and Grafana visualises.

12. Limitations and next steps

- Only 303 rows, so test metrics have high variance.
- No automatic retraining or drift detection; the next step would be data-drift metrics and a scheduled retraining workflow.
- Model is a decision-support demo, not a medical device.

13. Demo video

Screen recording (about 5.5 minutes) of the whole pipeline running end to end: data download and cleaning, EDA, model training, MLflow runs, lint and unit tests, the GitHub Actions runs, Docker image build and container run with `/health` and `/predict` calls, and the Prometheus and Grafana monitoring stack with live traffic:
https://drive.google.com/file/d/14Asgfh7xBR8TSk25dNophkGbxzHgQu49/view?usp=sharing

The same file is also in the repository at `demo/pipeline_demo.mp4`.